Order Management System

React Frontend Requirement Document

Version 1.0

1. Project Overview

The React frontend is the client application for the Order Management System (OMS). It provides browser-based screens for customer management, product and inventory viewing, order creation and management, and customer notifications.

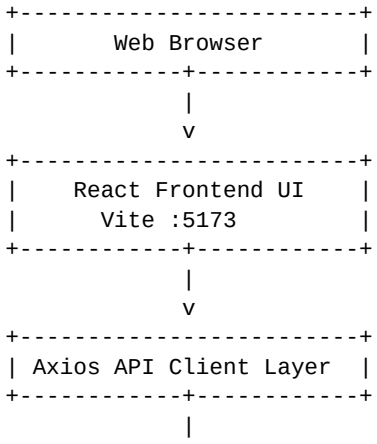
The existing OMS backend consists of four independent business services: Customer Service, Inventory Service, Order Service, and Notification Service. All external frontend requests must be sent through the API Gateway on port 8080. The frontend must not directly connect to any service database and must not directly call individual service instance ports.

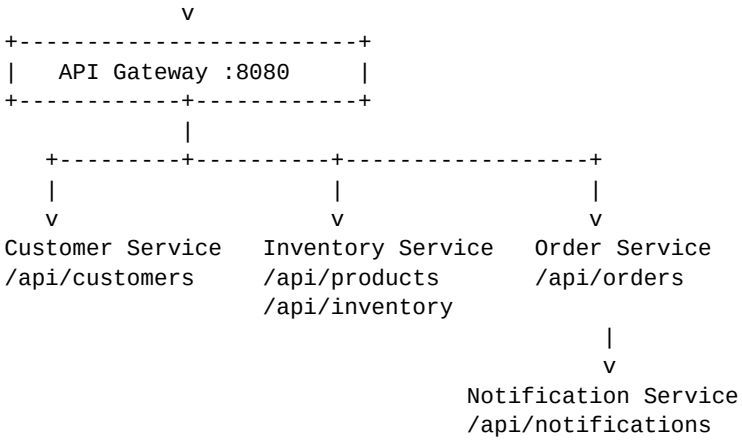
Frontend-specific technology choices in this document are implementation decisions for the React client. Backend API paths and business rules are aligned with the OMS Microservices Requirement Document Version 1.0.

2. Technology Stack

Area	Technology
Programming Language	JavaScript (ES6+)
Frontend Framework	React
Build Tool	Vite
Routing	React Router
HTTP Client	Axios
State Management	React Context API / component state
Form Handling	React Hook Form
Validation	Yup or equivalent client-side validation
UI / Styling	Bootstrap and CSS
User Feedback	React Toastify
Package Manager	npm
API Communication	REST APIs through API Gateway
Development Server	Vite development server (:5173 by default)
Backend Entry Point	API Gateway :8080
Containerization	Docker
Source Control	Git

3. High-Level Frontend Architecture





The React application acts only as an external client. Service discovery, load balancing, circuit breaking, Kafka processing, Redis caching, and database ownership remain backend responsibilities.

4. Frontend Module List

Module	Responsibility	Primary Backend APIs
Dashboard	Main navigation and entry screen	Optional; no mandatory backend API
Customer Management	Create, search, view, update, status update and delete customers	/api/customers/**
Product / Inventory	Display products and available stock	/api/products/**, /api/inventory/**
Order Management	Create orders, view orders/status and cancel orders	/api/orders/**
Notification Management	View notifications and update read state	/api/notifications/**

5. Customer Management

Responsibility: The Customer module provides frontend screens for managing customer information. Customer data is owned by Customer Service; the React application must use the Customer Service APIs exposed through the API Gateway.

5.1 Customer Screens

Screen	React Route	Purpose
Customer Search / List	/customers	Search or locate customer records
Create Customer	/customers/create	Create a new customer
Customer Details	/customers/:id	View customer information
Edit Customer	/customers/:id/edit	Update customer information

5.2 Customer APIs

Method	Endpoint	Frontend Usage
POST	/api/customers	Create customer
GET	/api/customers/{id}	Get customer
GET	/api/customers?email={email}	Find customer by email
PUT	/api/customers/{id}	Update customer
PATCH	/api/customers/{id}/status	Update customer status
DELETE	/api/customers/{id}	Delete customer

5.3 Create Customer

```
POST /api/customers

{
  "name": "Anil Kumar",
  "email": "anil@test.com",
  "phone": "9876543210"
}
```

Expected response fields from the backend requirement include id, name, email, phone and status. The frontend should display a success message and navigate to the created customer details after a successful response.

5.4 Customer Form Validation

- Name is required.
- Email is required and must be in valid email format.
- Phone should use the format accepted by the backend DTO.
- Server validation errors must be displayed to the user; client-side validation must not replace backend validation.

5.5 Customer Status

The frontend may provide ACTIVE/INACTIVE actions where supported by the backend implementation. The source requirement defines the PATCH endpoint but does not define its request payload; the React payload must match the actual Customer Service DTO.

6. Product and Inventory Module

Responsibility: Display products and their current available inventory. Inventory Service owns product and inventory data. The frontend must use API Gateway routes and must not access inventory_db directly.

6.1 Product / Inventory Screens

Screen	Route	Purpose
Product List	/products	Display all products
Product Details	/products/:id	Display product information
Create Product	/products/create	Create product where exposed to frontend users
Inventory Details	/products/:id/inventory	Display current available quantity

6.2 Product / Inventory APIs

Method	Endpoint	Frontend Usage
POST	/api/products	Create product
GET	/api/products	Get all products
GET	/api/products/{id}	Get product
GET	/api/inventory/{productId}	Get available inventory
PUT	/api/inventory/{productId}/stock	Update stock

6.3 Get Inventory

```
GET /api/inventory/5001

{
  "productId": 5001,
  "availableQuantity": 10
}
```

The frontend may display IN STOCK when availableQuantity is greater than zero and OUT OF STOCK when it is zero. These are presentation labels; the authoritative quantity is returned by Inventory Service.

7. Order Management

Responsibility: Order Management is the primary frontend business module. It allows a customer to select products, enter quantities, submit an order, view order details/status and cancel an order. The backend Order Service remains responsible for customer validation, inventory validation, status calculation and Kafka event publication.

7.1 Order Screens

Screen	Route	Purpose
Create Order	/orders/create	Submit an order request
Order Details	/orders/:id	View complete order
Customer Orders	/customers/:customerId/orders	View orders for a customer
Order Status	/orders/:id/status	Display current status
Order List	/orders	Optional navigation/search screen using available backend filters

7.2 Order APIs

Method	Endpoint	Frontend Usage
POST	/api/orders	Create order
GET	/api/orders/{id}	Get order
GET	/api/orders?customerId={id}	Get customer orders
GET	/api/orders/{id}/status	Get order status
PUT	/api/orders/{id}/cancel	Cancel order

7.3 Create Order Request

```
POST /api/orders

{
  "customerId": 101,
  "items": [
    {
      "productId": 5001,
      "quantity": 2
    },
    {
      "productId": 5002,
      "quantity": 1
    }
  ]
}
```

7.4 Order Creation Rules - Frontend Handling

- Frontend collects customerId and order items and sends them to POST /api/orders.
- Frontend must not independently decide whether an order is CONFIRMED or REJECTED.
- Order Service validates that the customer exists and is ACTIVE.
- Order Service checks inventory for every requested order item.
- If all requested quantities are available, Order Service creates the order with status CONFIRMED.
- If any item has insufficient inventory, the order is REJECTED.
- After backend order processing, the frontend displays the returned order status and any error/reason returned by the service.

7.5 Order Status

Status	Frontend Meaning
PENDING	Initial processing state
CONFIRMED	Customer and inventory validation succeeded
REJECTED	Order could not be confirmed, for example because of insufficient inventory
CANCELLED	A previously created order was subsequently cancelled

7.6 Order Status Flow

```
PENDING
   |
   v
Backend checks Customer
   |
   v
Backend checks Inventory
   |-------------------------|
Available                 Not Available
   |                         |
   v                         v
CONFIRMED                 REJECTED
   |
   | User requests Cancel
   v
CANCELLED
```

7.7 Cancel Order

The frontend should show a confirmation dialog before PUT /api/orders/{id}/cancel. The backend is the final authority for whether cancellation is valid. On success, the frontend should update the displayed order status to the status returned by Order Service.

8. Frontend-to-Backend Communication

All frontend REST calls must use the API Gateway base URL. The React application must not use hardcoded service instance URLs such as :8071, :8081, :8091 or :8101.

Frontend Caller	Gateway API	Purpose
Customer UI	/api/customers/**	Customer operations
Product UI	/api/products/**	Product operations
Inventory UI	/api/inventory/**	Inventory reads/updates
Order UI	/api/orders/**	Order operations
Notification UI	/api/notifications/**	Notification operations

9. Error and Service-Unavailable Handling

The frontend must handle validation errors, not-found responses, business-rule failures and temporary downstream service failures without exposing stack traces to the user.

Condition	Frontend Behaviour
400 / validation error	Display field-level or request-level validation message
404 / not found	Display resource-not-found message
Order rejected	Display returned REJECTED status and backend reason when provided
Customer not active / invalid	Display backend failure message and do not report success
Inventory service unavailable	Display a temporary service unavailable message

Unexpected 5xx error	Display generic server error and preserve details only in developer logging

The backend requirement specifies circuit breaker protection for Order Service calls to Customer and Inventory Services and gives INVENTORY_SERVICE_UNAVAILABLE as an example failure response. The frontend should render such backend failures clearly.

10. Notification Management

Responsibility: Display customer notifications created by Notification Service after it consumes Order Service Kafka events. The React client only reads and updates notification state using REST APIs; it does not consume Kafka directly.

10.1 Notification Screens

Screen	Route	Purpose
Notification List	/notifications	Display notifications for a customer
Notification Details	/notifications/:id	View one notification

10.2 Notification APIs

Method	Endpoint	Frontend Usage
GET	/api/notifications/{id}	Get notification
GET	/api/notifications?customerId={id}	Get customer notifications
PUT	/api/notifications/{id}/read	Mark notification as read
PUT	/api/notifications/read-all?customerId={id}	Mark all customer notifications as read

10.3 Notification Presentation

- Unread notifications should be visually distinct from read notifications.
- Clicking a notification may open its details and call the mark-as-read API when appropriate.
- Mark All As Read should call the backend and then update local UI state from the returned response.
- The frontend should display notification title, message, orderId when available, type, read state and created time when these fields are returned by the backend.

11. Backend Kafka Events and Frontend Impact

Kafka is a backend asynchronous communication mechanism. The React frontend does not publish or consume the order-events topic. Order Service publishes events and Notification Service consumes them. The frontend observes the result through Notification Service REST APIs.

```
Order created in React
        |
        v
POST /api/orders
        |
        v
Order Service
        |
        v
Kafka: order-events
        |
        v
Notification Service
        |
        v
GET /api/notifications?customerId={id}
```

```
        |
        v
React Notification UI
```

12. API Gateway Configuration for Frontend

The frontend API base URL should point to the API Gateway. A Vite environment variable should be used so the URL can differ by environment.

```
# .env
VITE_API_BASE_URL=http://localhost:8080


// src/api/axiosClient.js
import axios from "axios";

const axiosClient = axios.create({
  baseURL: import.meta.env.VITE_API_BASE_URL,
  headers: {
    "Content-Type": "application/json"
  }
});

export default axiosClient;
```

CORS must be configured in the API Gateway/backend deployment so that the React origin can call the Gateway during local development and deployment.

13. React Routing

Route	Component	Backend Interaction
/	Dashboard	None required
/customers	CustomerList/Search	Customer APIs
/customers/create	CustomerCreate	POST /api/customers
/customers/:id	CustomerDetails	GET /api/customers/{id}
/customers/:id/edit	CustomerEdit	PUT /api/customers/{id}
/products	ProductList	GET /api/products
/products/:id	ProductDetails	GET /api/products/{id}
/products/:id/inventory	InventoryDetails	GET /api/inventory/{productId}
/orders/create	CreateOrder	POST /api/orders
/orders/:id	OrderDetails	GET /api/orders/{id}
/customers/:customerId/orders	CustomerOrders	GET /api/orders?customerId={id}
/notifications	NotificationList	GET /api/notifications?customerId={id}
/notifications/:id	NotificationDetails	GET /api/notifications/{id}

14. Frontend State Management

For the scope defined in the backend requirement, React component state and Context API are sufficient. Global state should be introduced only where multiple screens need shared information.

State	Recommended Location	Purpose
Current customer selection	Context or route state	Reuse customerId across order and notification screens
Temporary cart/order items	CartContext	Build POST /api/orders request before submission
Notifications	Component state / NotificationContext	Display and update read state
Loading and errors	Per page/component	Control API feedback

Product list	Product page state	Display data from Inventory Service

The temporary cart is a frontend-only convenience. No Cart Service or cart database is defined in the source OMS requirement.

15. Form Validation and User Feedback

Area	Frontend Requirement
Customer forms	Validate required fields and basic format before submit
Order quantity	Must be a positive integer before submit
Product selection	At least one order item should be selected before order submission
API loading	Disable duplicate submit actions while request is in progress
Success	Display success toast/message after confirmed API success
Failure	Display backend message/reason where safe and available
Confirmation	Confirm destructive actions such as customer deletion or order cancellation

Client-side validation improves usability but does not replace backend validation or backend business rules.

16. Frontend Project Structure

```
order-management-ui/
|
+-- public/
+-- src/
|   +-- api/
|   |   +-- axiosClient.js
|   |   +-- customerApi.js
|   |   +-- productApi.js
|   |   +-- inventoryApi.js
|   |   +-- orderApi.js
|   |   +-- notificationApi.js
|   |
|   +-- components/
|   |   +-- Navbar.jsx
|   |   +-- LoadingSpinner.jsx
|   |   +-- ErrorMessage.jsx
|   |   +-- ConfirmDialog.jsx
|   |   +-- OrderStatusBadge.jsx
|   |   +-- ProductCard.jsx
|   |   +-- NotificationItem.jsx
|   |
|   +-- pages/
|   |   +-- dashboard/Dashboard.jsx
|   |   +-- customer/
|   |   |   +-- CustomerList.jsx
|   |   |   +-- CustomerCreate.jsx
|   |   |   +-- CustomerDetails.jsx
|   |   |   +-- CustomerEdit.jsx
|   |   +-- product/
|   |   |   +-- ProductList.jsx
|   |   |   +-- ProductDetails.jsx
|   |   |   +-- ProductCreate.jsx
|   |   |   +-- InventoryDetails.jsx
|   |   +-- order/
|   |   |   +-- CreateOrder.jsx
|   |   |   +-- OrderDetails.jsx
|   |   |   +-- CustomerOrders.jsx
```

```
|   |   +-- notification/
|   |       +-- NotificationList.jsx
|   |       +-- NotificationDetails.jsx
|   |
|   +-- context/
|   |   +-- CartContext.jsx
|   |   +-- CustomerContext.jsx
|   |   +-- NotificationContext.jsx
|   |
|   +-- utils/
|   |   +-- constants.js
|   |   +-- formatCurrency.js
|   |   +-- formatDate.js
|   |
|   +-- App.jsx
|   +-- main.jsx
|   +-- index.css
|
+-- .env
+-- package.json
+-- vite.config.js
```

17. End-to-End Frontend Order Flow

17.1 Successful Order

```
User
 |
 v
React Frontend
 |
 +-- Select / identify Customer
 |
 +-- Browse Products
 |
 +-- Enter Quantities
 |
 v
POST /api/orders
 |
 v
API Gateway :8080
 |
 v
Order Service
 |
 +-- REST -> Customer Service -> Customer ACTIVE
 |
 +-- REST -> Inventory Service -> Stock Available
 |
 v
CONFIRMED
 |
 v
Return OrderResponse to React
 |
 v
Display confirmed order
 |
 +--> Later load Notifications through REST
```

17.2 Rejected Order

```
React Frontend
 |
 v
POST /api/orders
 |
 v
Order Service
 |
 +-- Customer -> Valid
 |
 +-- Inventory -> Insufficient
 |
 v
REJECTED
 |
 v
Return rejected order / reason
 |
 v
```

```
React displays rejection to user
```

18. Frontend Non-Functional Requirements

Requirement	Description
Usability	Provide clear navigation, labels, validation and user feedback.
Responsiveness	Pages should support normal desktop and mobile-width browser layouts.
API Isolation	Use a centralized API client and call only the API Gateway base URL.
Fault Handling	Gracefully handle unavailable backend services and failed requests.
Consistency	Use reusable components for status badges, loaders, errors and dialogs.
Security	Do not store secrets in source code or frontend environment variables. Authentication is not defined in the source scope.
Maintainability	Organize pages, API modules, components, context and utilities separately.
Logging	Use browser console logging only for development; avoid exposing sensitive backend details to users.
Performance	Avoid unnecessary duplicate API calls and use local state appropriately.
Accessibility	Use semantic labels, keyboard-friendly controls and accessible form feedback.

19. Frontend Testing Requirements

Test Area	Required Tests
Customer	Create, get, search, update, status change and delete flows
Products	Load all products and individual product details
Inventory	Load stock for valid and invalid product IDs
Orders	Create confirmed order, rejected order, get details, status and cancellation
Notifications	Load list/details, mark one read, mark all read
Gateway	Verify all requests use :8080 Gateway and no direct service port
Failure handling	Validate 4xx, 5xx and downstream-unavailable UI behaviour
Forms	Required fields, invalid email, invalid quantities and duplicate submit prevention
Routing	Direct navigation and parameterized routes
Responsive UI	Basic desktop/tablet/mobile layout checks

20. Docker and Deployment Requirements

The original OMS requires all application components to be containerized. The React frontend can be added as an additional container in the complete stack.

```
Docker Compose
|
+-- react-frontend
+-- api-gateway
+-- service-registry
+-- customer-service-1
+-- customer-service-2
+-- order-service-1
+-- order-service-2
```

```
+-- inventory-service-1
+-- inventory-service-2
+-- notification-service-1
+-- notification-service-2
+-- postgres
+-- redis
+-- kafka
```

For production, the React application should be built using npm run build and served by a lightweight web server such as Nginx inside its container. The runtime API Gateway URL must be configured appropriately for the deployment environment.

21. Frontend Technology Coverage

Technology	Usage in Frontend
React	Component-based user interface
Vite	Development and production build tooling
React Router	Client-side navigation
Axios	REST communication with API Gateway
Context API	Shared customer/cart/notification state where needed
React Hook Form	Form state and submit handling
Yup / validation library	Client-side input validation
Bootstrap / CSS	Responsive UI styling
React Toastify	Success/error notifications
Docker	Frontend containerization
API Gateway	Single backend entry point

22. Key Frontend Business Rules

- All external frontend API calls must go through the API Gateway.
- The frontend must never access a microservice database directly.
- The frontend must not call backend service instance ports directly.
- Customer must exist and be ACTIVE before an order can be confirmed; this validation is performed by Order Service.
- Every requested product quantity must be available before an order can be confirmed; this validation is performed by Order Service.
- The frontend must display the backend-provided order status: PENDING, CONFIRMED, REJECTED or CANCELLED.
- If any order item has insufficient inventory, the backend rejects the order and the frontend displays the returned failure result.
- Order cancellation must be sent to PUT /api/orders/{id}/cancel and validated by Order Service.
- The frontend does not communicate with Kafka directly.
- The frontend does not interact with Redis directly.
- Notification data is obtained through Notification Service REST APIs.
- Frontend cart/order-item state is temporary client state; no Cart Service is defined by the source requirement.
- Authentication, payment, shipment and delivery modules are outside the supplied OMS requirement and should not be introduced unless requirements are expanded.

23. Recommended Frontend Implementation Sequence

1. Create the React application using Vite.

2. Install React Router, Axios, Bootstrap, React Toastify and form-validation dependencies.

3. Create the application layout and navigation.

4. Configure VITE_API_BASE_URL to the API Gateway.

5. Create the centralized Axios client and API modules.

6. Implement Customer create/search/details/update screens.

7. Implement Product List and Product Details screens.

8. Implement Inventory display.

9. Implement CartContext for temporary order items.

10. Implement Create Order and connect POST /api/orders.

11. Implement Order Details, Customer Orders, Order Status and Cancel Order.

12. Implement Notification List and Notification Details.

13. Implement mark-as-read and mark-all-as-read actions.

14. Add shared loading, error, toast and confirmation components.

15. Add form validation and duplicate-submit prevention.

16. Test all calls through API Gateway.

17. Validate CORS for the React development origin.

18. Add frontend Dockerfile and integrate with Docker Compose.

19. Perform end-to-end testing for confirmed, rejected and cancelled order flows.